

LAPORAN TUGAS AKHIR

“DSS Evakuasi Bencana”



INSTIKI
INSTITUT BISNIS DAN
TEKNOLOGI INDONESIA

OLEH:

I KETUT KAYANA PRADHITYA

I GEDE EGA SANJAYA PUTRA

DEWA GEDE REIHAN WAISWANA

DAFTAR ISI

BAB I	3
PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Rumusan masalah.....	4
1.3 Tujuan	4
1.4 Manfaat	4
BAB II.....	5
DASAR TEORI	5
2.1 Struktur data graph.....	5
2.2 DSS (Decision Support System)	5
2.3 Algoritma graph yang digunakan.....	5
BAB III.....	6
ANALISIS DAN PERANCANGAN.....	6
3.1 Analisis masalah.....	6
3.2 Desain graph.....	6
3.3 Flowchart (ringkas, deskriptif).....	6
3.4 Daftar Peralatan dan Bahan.....	6
3.5 Daftar Total Biaya.....	7
BAB IV	8
IMPLEMENTASI.....	8
4.1 Implementasi program.....	8
4.2 Penjelasan kode (inti)	8
4.3 Tampilan sistem (deskripsi)	8
BAB V.....	10
PENGUJIAN DAN ANALISIS	10
5.1 Skenario pengujian.....	10
5.2 Analisis hasil	10
5.3 Kompleksitas algoritma.....	11
BAB VI	12
PENUTUP.....	12
6.1 Kesimpulan	12
6.2 Saran Pengembangan	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam situasi bencana, pemilihan jalur evakuasi yang tepat sangat penting untuk membantu masyarakat mencapai lokasi aman dengan cepat dan efisien. Namun, banyaknya alternatif jalur serta adanya area yang berisiko dapat menyulitkan proses pengambilan keputusan. Oleh karena itu, diperlukan suatu sistem yang dapat membantu menentukan jalur evakuasi terbaik berdasarkan data yang tersedia.

Pada proyek ini, jaringan lokasi evakuasi dimodelkan dalam bentuk graph yang terdiri atas node dan edge. Setiap node merepresentasikan suatu lokasi dengan kategori tertentu, seperti area aman, area bahaya, fasilitas medis, atau bangunan penting. Sementara itu, setiap edge merepresentasikan jalur penghubung antar lokasi yang memiliki bobot dan status keamanan. Sistem kemudian memanfaatkan algoritma Dijkstra untuk menghitung jalur terbaik dari lokasi awal menuju lokasi tujuan dengan mempertimbangkan bobot jalur yang tersedia.

Untuk mempermudah pengguna dalam memahami hasil analisis, sistem dikembangkan menggunakan framework Streamlit yang menyediakan antarmuka interaktif berbasis web. Selain itu, visualisasi data dilakukan menggunakan peta interaktif sehingga pengguna dapat melihat lokasi, area berbahaya, serta jalur evakuasi yang direkomendasikan secara langsung. Dengan adanya sistem ini, proses pencarian jalur evakuasi menjadi lebih mudah, cepat, dan informatif, sehingga dapat membantu pengguna dalam mengambil keputusan berdasarkan data yang tersedia.

1.2 Rumusan Masalah

Berikut Rumusan masalah yang ada:

1. Bagaimana memodelkan jaringan lokasi dan jalur evakuasi sebagai graph yang fleksibel (mendukung status aman/berbahaya)?
2. Bagaimana mencari rute evakuasi dengan bobot minimum dari titik awal ke titik tujuan?
3. Bagaimana menampilkan hasil analisis dalam antarmuka interaktif dan peta yang mudah dipahami?

1.3 Tujuan

1. Mengimplementasikan sistem pendukung keputusan (DSS) jalur evakuasi berbasis struktur data graph.
2. Menerapkan algoritma Dijkstra untuk menemukan jalur dengan total bobot minimum.
3. Menyediakan input data node/edge, visualisasi graph di peta, hasil rekomendasi rute, serta indikator area berbahaya.

1.4 Manfaat

- A. Membantu pemangku kepentingan (BPBD, relawan, masyarakat) memilih rute evakuasi yang relatif aman dan efisien.
- B. Menjadi contoh nyata implementasi konsep graph dalam DSS spasial.
- C. Dapat diperluas dengan data real-time, analisis tambahan, dan integrasi layanan rute jalan sebenarnya.

BAB II

DASAR TEORI

2.1 Struktur data graph

1. Graph $G = (V, E)$ merepresentasikan sekumpulan node/vertex V (lokasi) dan edge E (jalur antar lokasi).
2. Pada sistem ini, node memiliki atribut: id, nama lokasi, tipe (Aman, Bahaya, Bangunan, Medis), serta koordinat lat-lon.
3. Edge memiliki atribut: source, target, weight (bobot/"jarak"), dan status (Aman/Bahaya).
4. Graph yang digunakan: directed/undirected dapat disesuaikan di builder; pada implementasi praktis rute jalan biasanya bersifat directed atau bidirectional simetris. Kode menyediakan builder graph (build_graph) dari file edges.csv.

2.2 DSS (Decision Support System)

- a. DSS memadukan data (nodes.csv, edges.csv), model/algoritma (Dijkstra), serta antarmuka (Streamlit + peta Folium) untuk menghasilkan rekomendasi rute.
- b. Fitur inti DSS di sini: input data, pemodelan graph, analisis rute minimum, pelabelan risiko, penyajian hasil

2.3 Algoritma graph yang digunakan

- A. Algoritma Dijkstra: mencari jarak/bobot minimum dari sumber ke semua node lain pada graph berbobot non-negatif
- B. Kompleksitas umum dengan priority queue: $O((V + E) \log V)$.
- C. Kesesuaian: bobot merepresentasikan "biaya" (jarak/waktu/risiko teragregasi) dan semua bobot diasumsikan non-negatif.

BAB III

ANALISIS DAN PERANCANGAN

3.1 Analisis masalah

1. Masukan: himpunan lokasi (V) dengan klasifikasi zona (Aman/Bahaya/Medis) dan jalur (E) dengan bobot dan status (Aman/Bahaya).
2. Keluaran: jalur evakuasi dari lokasi awal ke titik tujuan dengan total bobot minimum, disertai peringatan jika rute melewati jalur berbahaya.
3. Kendala: ketika opsi “hindari area berbahaya” aktif, jalur berstatus “Bahaya” diupayakan tidak dilalui; jika tak ada alternatif, sistem tetap memberi rute dengan peringatan.

3.2 Desain graph

1. Node: id unik; atribut lat, lon; tipe kategori (Aman/Bahaya/Medis/Bangunan).
2. Edge: source, target, weight, status. Weight dapat diartikan jarak atau biaya perjalanan.
3. Representasi internal: adjacency list (melalui builder graph dari edges.csv) untuk efisiensi penyimpanan dan traversal.

3.3 Flowchart (ringkas, deskriptif)

Mulai → Muat nodes.csv & edges.csv → Bangun graph (filter sesuai opsi hindari bahaya) → Pilih lokasi_awal & tujuan → Jalankan Dijkstra → Dapatkan best_path & total_cost → Cek apakah path mengandung edge “Bahaya” → Tampilkan hasil & peringatan → Tampilkan peta dan rute mengikuti jalan asli via layanan routing → Selesai.

3.4 Daftar Peralatan dan Bahan

- **Aktor: Pengguna (koordinator, relawan).**
- **Skenario utama: Pengguna memilih jenis bencana, opsi hindari bahaya, lokasi awal, dan tujuan; sistem menghitung dan menampilkan rute rekomendasi beserta metrik dan visualisasi peta; pengguna dapat memperbarui data node/edge dari sidebar.**

3.5 Daftar Total Biaya

- **Node:** {id: string, lokasi: string, tipe: {Aman, Bahaya, Bangunan, Medis}, lat: float, lon: float}.
- **Edge:** {source: node_id, target: node_id, weight: int/float, status: {Aman, Bahaya}}.

BAB IV

IMPLEMENTASI

4.1 Implementasi program

1. Bahasa/Framework: Python + Streamlit untuk UI, Folium/streamlit-folium untuk peta, openrouteservice untuk mengikuti jaringan jalan, pandas untuk I/O CSV.
2. File data: dataset/nodes.csv dan dataset/edges.csv.
3. Modul utama:
 - `graph.graph_builder.build_graph`: membangun graph dari edges.csv; parameter `avoid_danger` untuk menyaring edge berbahaya.
 - `algorithms.dijkstra.dijkstra`: mengembalikan (`total_cost`, `best_path`) untuk pasangan (`lokasi_awal`, `tujuan`).

4.2 Penjelasan kode (inti)

1. Sidebar:
 - Pilihan jenis bencana (Gempa/Banjir/Tsunami) sebagai konteks filter/aturan builder.
 - Checkbox “Hindari Area Berbahaya”.
 - Input `lokasi_awal` dan `tujuan` dari daftar node id.
 - Fitur tambah node dan tambah edge yang langsung menyimpan perubahan ke CSV.
2. Statistik: jumlah node, jumlah edge, jumlah zona bahaya (node tipe Bahaya), jumlah jalur aman (edge status Aman).
3. Proses Dijkstra: tombol “Cari Jalur Terbaik” mengeksekusi `dijkstra(graph, lokasi_awal, tujuan)` dan menyimpan hasil di `st.session_state`.
4. Validasi bahaya: fungsi `path_has_danger` memeriksa apakah path melewati edge berstatus “Bahaya” dan menampilkan peringatan sesuai opsi `hindari_bahaya`.
5. Peta:
 - Marker: warna berdasarkan tipe node (Bahaya=merah, Aman=hijau, Medis=oranye, lainnya=biru).
 - HeatMap: intensitas pada node bertipe Bahaya.
 - Rute mengikuti jalan asli: memanggil `openrouteservice.directions` dengan urutan koordinat path, menggambar PolyLine biru di atas peta.
 - MiniMap dan Fullscreen plugin untuk pengalaman navigasi.
6. Tampilan:
 - Judul, deskripsi singkat, metrik, hasil jalur, total bobot, peta interaktif.

4.3 Tampilan sistem (deskripsi)

Halaman utama: judul “DSS Jalur Evakuasi Bencana”, metrik ringkas (Jumlah Lokasi/Jalur, Zona Bahaya, Jalur Aman), panel sidebar input, area peta besar dengan marker, heatmap bahaya, dan polyline rute.

BAB V

PENGUJIAN DAN ANALISIS

5.1 Skenario pengujian

1. Pengujian jalur aman tersedia:

- Data memiliki jalur alternatif “Aman” dengan bobot lebih kecil dari rute yang mengandung bahaya.
- Ekspektasi: Dijkstra memilih rute aman; tidak muncul peringatan bahaya.

2. Pengujian hanya tersedia jalur berbahaya:

- Semua jalur atau sisi kritis berstatus “Bahaya”.
- Ekspektasi: Jika “Hindari Area Berbahaya” aktif namun tak ada alternatif, sistem tetap menampilkan rute terbaik yang ada dan memberikan peringatan khusus.

3. Pengujian update data:

- Tambahkan node dan edge baru via sidebar; simpan ke CSV.
- Ekspektasi: metrik bertambah, builder memuat ulang graph, rute dapat berubah sesuai data baru.

4. Pengujian rute multi-segmen:

- Path terdiri dari beberapa node, layanan routing eksternal (openrouteservice) sukses memberi geometri jalan.
- Ekspektasi: PolyLine muncul, marker awal/akhir sesuai, tanpa error.

5. Pengujian kegagalan routing:

- Matikan koneksi internet atau API key tidak valid.
- Ekspektasi: aplikasi tetap memberikan hasil jalur (dari Dijkstra) namun menampilkan pesan error “Routing error: ...” tanpa crash.

5.2 Analisis hasil

- Dijkstra efektif menemukan jalur dengan bobot total minimum pada graph berbobot non-negatif.

- Opsi “hindari bahaya” berfungsi sebagai constraint lunak: tetap memberi hasil dengan peringatan bila tidak ada alternatif aman.
- Visualisasi heatmap membantu mengidentifikasi konsentrasi risiko; marker berwarna memudahkan interpretasi.

5.3 Kompleksitas algoritma

Dijkstra dengan priority queue: $O((V + E) \log V)$. Pada skala kota menengah dengan optimasi adjacency list, masih responsif untuk antarmuka interaktif.

BAB VI

PENUTUP

6.1 Kesimpulan

- Sistem berhasil memodelkan jaringan evakuasi sebagai graph, menerapkan Dijkstra untuk rekomendasi jalur minimum, dan menyajikan hasil secara interaktif di peta.
- Integrasi status bahaya pada edge dan tipe node meningkatkan relevansi keputusan saat bencana.

6.2 Saran Pengembangan

- Terapkan multi-kriteria weight (gabungan jarak, waktu tempuh, elevasi, risiko genangan) dan A* untuk percepatan pencarian.
- Tambah analisis centrality (menemukan bottleneck), community detection/cluster untuk zonasi evakuasi.
- Integrasi data real-time (banjir/kemacetan), graph database (mis. Neo4j), dan cache rute.
- Implementasi multi-user, API publik, dan deployment online.
- Validasi lapangan serta uji beban untuk skala besar.

Lampiran poin syarat tambahan yang telah terpenuhi dari kode:

- Representasi graph: adjacency list dari edges.csv; mendukung edge weighted dan status (Aman/Bahaya).
- Algoritma graph utama: Dijkstra.

- Fitur minimum:
 - Input data node dan edge (melalui sidebar).
 - Visualisasi graph pada peta (marker, heatmap, polyline rute).
 - Proses analisis keputusan (pemilihan jalur minimum, opsi hindari bahaya).
 - Menampilkan hasil rekomendasi dan total bobot, serta peringatan bahaya.
 - Antarmuka interaktif (Streamlit + Folium).
- Teknologi: Python, Streamlit, Folium, pandas, openrouteservice.